

Materi CRUD Produk dengan Laravel

Daftar Isi

1. Gambaran Umum
 2. Struktur File
 3. Langkah Implementasi
 - Langkah 1 – Migrasi Database
 - Langkah 2 – Model Product
 - Langkah 3 – Controller ResourceController
 - Langkah 4 – Routing Resource
 - Langkah 5 – Method index (Read All)
 - Langkah 6 – Method create & store (Create)
 - Langkah 7 – Method show (Read One)
 - Langkah 8 – Method edit & update (Update)
 - Langkah 9 – Method destroy (Delete)
 - Langkah 10 – Views Blade
 4. Form Validation
 - Aturan Validasi yang Digunakan
 - Pesan Error Kustom
 - Menampilkan Error di View
 - Mempertahankan Input Lama (old())
 5. Method Spoofing (PUT & DELETE)
 6. Flash Message
 7. Ringkasan Route Resource
-

Gambaran Umum

CRUD adalah singkatan dari **Create, Read, Update, Delete** — empat operasi dasar pada data.

Laravel menyediakan **Resource Controller** yang secara otomatis memetakan tujuh method standar ke operasi CRUD, sehingga tidak perlu mendefinisikan route satu per satu.

Struktur File

```
app/
  Http/Controllers/
    ProductController.php      ← logika CRUD
  Models/
    Product.php                ← Eloquent model
database/
  migrations/
    2026_05_05_..._create_products_table.php
  seeders/
    ProductSeeder.php
resources/views/produk/
  index.blade.php              ← daftar produk (Read All)
  create.blade.php              ← form tambah produk (Create)
  edit.blade.php                ← form edit produk (Update)
  detail.blade.php              ← detail produk (Read One)
  search.blade.php              ← pencarian produk
routes/
  web.php                       ← definisi route resource
```

Langkah Implementasi

Langkah 1 – Migrasi Database

Tabel `products` dibuat dengan perintah:

```
php artisan make:migration create_products_table
```

Struktur kolom tabel `products`:

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name', 100);
    $table->decimal('price', 10, 2);
    $table->text('description')->nullable();
    $table->enum('status', ['new', 'used'])->default('new');
    $table->boolean('is_active')->default(true);
    $table->date('release_date')->nullable();
    $table->timestamps();
});
```

Jalankan migrasi:

```
php artisan migrate
```

Langkah 2 – Model Product

File: app/Models/Product.php

```
class Product extends Model
{
    use HasFactory;

    protected $table = 'products';

    // $fillable wajib didefinisikan agar mass assignment (Product::create())
    // dapat berjalan. Tanpa ini, Laravel melempar MassAssignmentException.
    protected $fillable = [
        'name', 'price', 'description', 'status', 'is_active', 'release_date',
    ];
}
```

\$fillable adalah whitelist kolom yang boleh diisi secara massal melalui `create()` atau `update()`. Ini adalah perlindungan terhadap serangan **Mass Assignment** (salah satu OWASP Top 10).

Langkah 3 – Controller ResourceController

Buat controller dengan flag `--resource` agar Laravel otomatis men-generate tujuh method:

```
php artisan make:controller ProductController --resource
```

Method yang di-generate:

Method	Fungsi
<code>index()</code>	Menampilkan daftar semua data
<code>create()</code>	Menampilkan form tambah data
<code>store()</code>	Menyimpan data baru ke DB
<code>show()</code>	Menampilkan detail satu data
<code>edit()</code>	Menampilkan form edit data
<code>update()</code>	Memperbarui data di DB
<code>destroy()</code>	Menghapus data dari DB

Langkah 4 – Routing Resource

File: routes/web.php

```
use App\Http\Controllers\ProductController;

Route::resource('/produk', ProductController::class);
```

Satu baris ini otomatis mendaftarkan **7 route** sekaligus. Lihat semua route dengan:

```
php artisan route:list
```

Langkah 5 – Method index (Read All)

```
public function index()
{
    $title    = "Daftar Produk";
    $products = Product::paginate(10); // ambil 10 data per halaman
    return view('produk.index', compact('title', 'products'));
}
```

Di view `index.blade.php`, tampilkan pagination dengan:

```
{{ $products->links() }}
```

Langkah 6 – Method create & store (Create)

create() hanya menampilkan form kosong:

```
public function create()
{
    $title = "Tambah Produk";
    return view('produk.create', compact('title'));
}
```

store() menerima data POST, memvalidasi, lalu menyimpan:

```
public function store(Request $request)
{
    $validated = $request->validate([...]); // validasi input
    $validated['is_active'] = $request->has('is_active') ? 1 : 0; // tangani checkbox
    Product::create($validated);           // simpan ke DB
    return redirect()->route('produk.index')
        ->with('success', 'Produk berhasil ditambahkan.');
```

Langkah 7 – Method show (Read One)

```
public function show(string $id)
{
    $title    = "Detail Produk";
    $product = Product::findOrFail($id); // 404 otomatis jika tidak ditemukan
    return view('produk.detail', compact('product', 'title'));
}
```

findOrFail() lebih aman dibanding *find()* karena secara otomatis melempar *ModelNotFoundException* (HTTP 404) jika data tidak ada, mencegah error tak terduga.

Langkah 8 – Method edit & update (Update)

edit() menampilkan form dengan data lama:

```
public function edit(string $id)
{
    $title    = "Edit Produk";
    $product = Product::findOrFail($id);
    return view('produk.edit', compact('product', 'title'));
}
```

update() memvalidasi lalu memperbarui data:

```
public function update(Request $request, string $id)
{
    $product = Product::findOrFail($id);
    $validated = $request->validate([...]);
    $validated['is_active'] = $request->has('is_active') ? 1 : 0;
    $product->update($validated);
    return redirect()->route('produk.index')
        ->with('success', 'Produk berhasil diperbarui.');
```

Langkah 9 – Method destroy (Delete)

```
public function destroy(string $id)
{
    $product = Product::findOrFail($id);
    $product->delete();
    return redirect()->route('produk.index')
        ->with('success', 'Produk berhasil dihapus.');
```

Langkah 10 – Views Blade

View	Route yang memanggil
index.blade.php	GET /produk
create.blade.php	GET /produk/create
edit.blade.php	GET /produk/{id}/edit
detail.blade.php	GET /produk/{id}
search.blade.php	GET /produk/search

Form Validation

Laravel menyediakan sistem validasi bawaan melalui method `$request->validate()`. Ketika validasi gagal, Laravel **secara otomatis** mengarahkan kembali ke halaman sebelumnya dan menyimpan semua pesan error di session.

Aturan Validasi yang Digunakan

```
$request->validate([
    'name'          => 'required|string|max:100',
    'price'         => 'required|numeric|min:0',
    'description'   => 'nullable|string',
    'status'        => 'required|in:new,used',
    'is_active'     => 'nullable|boolean',
    'release_date'  => 'nullable|date',
]);
```

Penjelasan setiap aturan:

Aturan	Fungsi
required	Field wajib diisi, tidak boleh kosong atau null
string	Nilai harus berupa string (teks)
max:100	Panjang string maksimal 100 karakter
numeric	Nilai harus berupa angka (integer atau desimal)
min:0	Nilai angka minimal 0 (tidak boleh negatif)
nullable	Field boleh bernilai null / tidak diisi
in:new,used	Nilai hanya boleh salah satu dari daftar yang ditentukan
boolean	Nilai harus <code>true</code> , <code>false</code> , <code>1</code> , <code>0</code> , <code>"true"</code> , atau <code>"false"</code>
date	Nilai harus berupa format tanggal yang valid (dapat diparse oleh PHP)

Pesan Error Kustom

Argumen kedua pada `validate()` adalah array pesan kustom untuk mengganti pesan default Laravel menjadi pesan berbahasa Indonesia:

```

$request->validate(
    [
        'name' => 'required|string|max:100',
        // ...
    ],
    [
        // Format: 'field.aturan' => 'pesan kustom'
        'name.required'    => 'Nama produk wajib diisi.',
        'name.max'         => 'Nama produk maksimal 100 karakter.',
        'price.required'   => 'Harga produk wajib diisi.',
        'price.numeric'    => 'Harga produk harus berupa angka.',
        'price.min'        => 'Harga produk tidak boleh negatif.',
        'status.required'  => 'Status produk wajib dipilih.',
        'status.in'        => 'Status produk harus new atau used.',
        'release_date.date'=> 'Format tanggal rilis tidak valid.',
    ]
);

```

Menampilkan Error di View

Tampilkan semua error sekaligus (ringkasan di atas form):

```

@if ($errors->any())
    <div class="alert alert-danger">
        <ul class="mb-0">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

```

Tampilkan error per field (di bawah input masing-masing):

```

<input type="text" name="name"
        class="form-control @error('name') is-invalid @enderror"
        value="{{ old('name') }}">

@error('name')
    <div class="invalid-feedback">{{ $message }}</div>
@enderror

```

Penjelasan direktif Blade:

Direktif

@error('field')

\$message

Fungsi

Blok ini hanya dirender jika field tersebut memiliki error

Variabel otomatis berisi pesan error untuk field saat ini

Direktif

Fungsi

<code>\$errors->any()</code>	Mengembalikan <code>true</code> jika ada error apapun setelah validasi gagal
<code>\$errors->all()</code>	Mengembalikan array semua pesan error dari semua field
<code>is-invalid</code>	Class Bootstrap 5 untuk menampilkan border merah pada input
<code>invalid-feedback</code>	Class Bootstrap 5 untuk menampilkan teks pesan error di bawah input

Mempertahankan Input Lama (old())

Ketika validasi gagal dan pengguna diarahkan kembali ke form, input yang sudah diisi sebelumnya tidak hilang berkat fungsi `old()`:

```
{{-- Mengisi ulang input teks --}}
<input type="text" name="name" value="{{ old('name') }}">

{{-- Pada form edit: old() mengambil nilai lama dari sesi validasi,
    atau jika tidak ada (pertama kali buka), gunakan nilai dari database --}}
<input type="text" name="name" value="{{ old('name', $product->name) }}">

{{-- Mengembalikan pilihan select --}}
<option value="new" {{ old('status', $product->status) === 'new' ? 'selected' : '' }}>Baru</option>

{{-- Mengembalikan state checkbox --}}
<input type="checkbox" name="is_active" value="1"
    {{ old('is_active', $product->is_active) ? 'checked' : '' }}>
```

*`old('field', $default)` — argumen pertama adalah nama field, argumen kedua (opsional) adalah nilai default jika tidak ada nilai lama di session. Sangat berguna pada form **edit** agar nilai dari database ditampilkan saat pertama kali membuka form.*

Method Spoofing (PUT & DELETE)

HTML form hanya mendukung method `GET` dan `POST`. Untuk method `PUT` (update) dan `DELETE` (hapus), Laravel menggunakan teknik **method spoofing** dengan menyisipkan hidden field `_method`:


```

{{-- Form Update --}}
<form action="{{ route('produk.update', $product->id) }}" method="POST">
    @csrf
    @method('PUT')    {{-- Dikirim sebagai _method=PUT --}}
    ...
</form>

{{-- Form Delete --}}
<form action="{{ route('produk.destroy', $item->id) }}" method="POST">
    @csrf
    @method('DELETE')    {{-- Dikirim sebagai _method=DELETE --}}
    <button type="submit">Hapus</button>
</form>

```

*@csrf wajib ada pada setiap form POST untuk mencegah serangan **Cross-Site Request Forgery (CSRF)**. Laravel memverifikasi token ini secara otomatis pada setiap request yang memodifikasi data.*

Flash Message

Setelah operasi CRUD berhasil, controller mengirim pesan sukses melalui session:

```

return redirect()->route('produk.index')
    ->with('success', 'Produk berhasil ditambahkan.');
```

Di view, tampilkan pesan tersebut:

```

@if (session('success'))
    <div class="alert alert-success alert-dismissible fade show">
        {{ session('success') }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
@endif

```

Ringkasan Route Resource

Hasil dari `Route::resource('/produk', ProductController::class):`

HTTP Method	URI	Method Controller	Nama Route	Fungsi
GET	/produk	index()	produk.index	Tampilkan daftar produk
GET	/produk/create	create()	produk.create	Tampilkan form tambah
POST	/produk	store()	produk.store	Simpan produk baru
GET	/produk/{produk}	show()	produk.show	Tampilkan detail produk

HTTP Method	URI	Method Controller	Nama Route	Fungsi
GET	/produk/{produk}/edit	edit()	produk.edit	Tampilkan form edit
PUT/PATCH	/produk/{produk}	update()	produk.update	Perbarui data produk
DELETE	/produk/{produk}	destroy()	produk.destroy	Hapus produk

Dengan menggunakan resource controller, kita dapat mengelola semua operasi CRUD dengan struktur yang rapi dan konsisten, serta memanfaatkan fitur-fitur Laravel seperti validasi, flash message, dan method spoofing dengan mudah.